

Wazuh PowerShell Detection Engineering

PowerShell monitoring, custom Wazuh alerting, and MITRE ATT&CK-mapped investigation

PROJECT DOCUMENTATION REPORT

Prepared by: Dhiraj Antre

Career track: SOC Analyst (entry-level)

Environment: Wazuh 4.14.5 · Windows 11 endpoint · Wazuh Manager on Kali Linux (VMware)

Repository: github.com/dhirajantre/wazuh-powershell-detection-engineering

Lab Safety: this project was performed in a controlled personal SOC home lab. The PowerShell activity used for detection validation was intentionally executed for security testing and detection-engineering purposes. No real-world malicious activity or unauthorized access was performed.

Executive Summary

This report documents a self-directed detection-engineering project built around Wazuh and Windows PowerShell Script Block Logging. A Windows 11 endpoint was connected to a Wazuh Manager, PowerShell activity was collected as telemetry, and a custom Wazuh rule was written to flag suspicious **Invoke-Expression (IEX)** usage - a technique commonly abused to execute obfuscated or remotely staged PowerShell code.

The detection was validated with a controlled test command, the resulting Wazuh alert was investigated end to end, and the activity was mapped to MITRE ATT&CK T1059.001 (Command and Scripting Interpreter: PowerShell). The event was classified as benign, authorized security testing.

Logging	Detection Rule	Alert	Investigation	MITRE Mapping
Enabled & verified	Custom rule 100100	Fired & verified	Completed	T1059.001

1. Project Overview

PowerShell is a legitimate Windows administration and automation framework, but it can also be abused by attackers to execute commands and scripts. In this project, a Windows 11 endpoint was connected to a locally hosted Wazuh Manager running inside a Kali Linux virtual machine. PowerShell Script Block Logging was enabled on the endpoint, and PowerShell Operational events were collected by the Wazuh Agent. A custom Wazuh detection rule was created to identify PowerShell Script Block content containing **Invoke-Expression** or **IEX**. A controlled PowerShell test was executed to validate the detection, and the resulting alert was investigated and mapped to MITRE ATT&CK T1059.001.

Objectives

- Configure Windows PowerShell logging
- Enable PowerShell Script Block Logging
- Connect a Windows endpoint to Wazuh
- Collect PowerShell Operational events
- Analyze PowerShell activity in Wazuh
- Develop a custom Wazuh detection rule
- Detect Invoke-Expression (IEX) usage
- Validate the detection with a controlled test
- Map the detection to MITRE ATT&CK
- Investigate the generated security alert
- Document the incident and evidence

2. Lab Environment

Component	Configuration
Endpoint OS	Windows 11 Pro
Endpoint Hostname	DHIRAJ-WIN11

4. PowerShell Logging Configuration

PowerShell Operational Log

The Windows PowerShell Operational event channel was verified before testing:

```
Get-WinEvent -ListLog "Microsoft-Windows-PowerShell/Operational" |  
Select-Object LogName, IsEnabled
```

The log channel was confirmed enabled.

Enable Script Block Logging

PowerShell Script Block Logging was enabled via Windows Registry policy:

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" `   
-Force | Out-Null  
  
Set-ItemProperty `   
-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" `   
-Name EnableScriptBlockLogging -Value 1
```

Verified with:

```
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
```

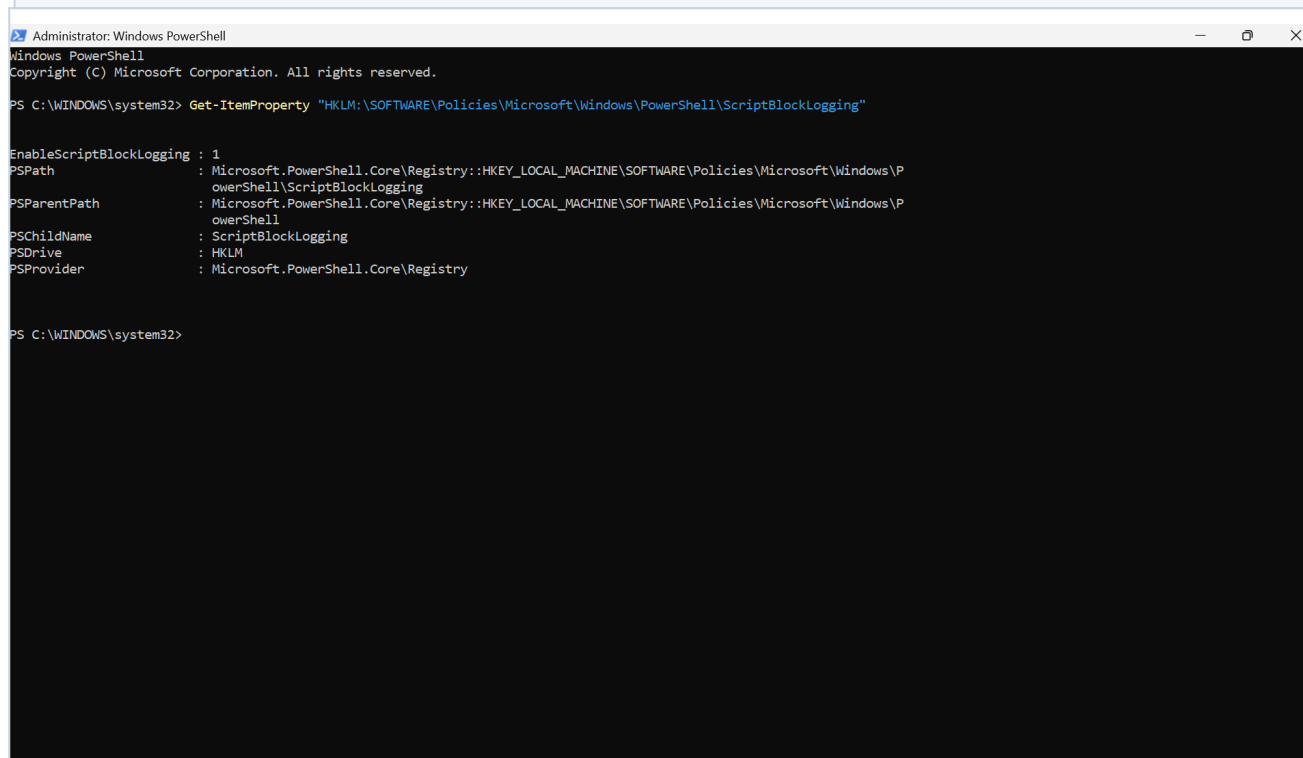


Figure 1. PowerShell registry check confirming `EnableScriptBlockLogging = 1`.

5. Wazuh Agent Deployment

The Windows endpoint was connected to the Wazuh Manager running inside the Kali Linux virtual machine at 192.168.47.128. Once registered, the agent (DHIRAJ-WIN11, version 4.14.5) reported as active in the Wazuh dashboard, alongside the manager's live alert severity summary.

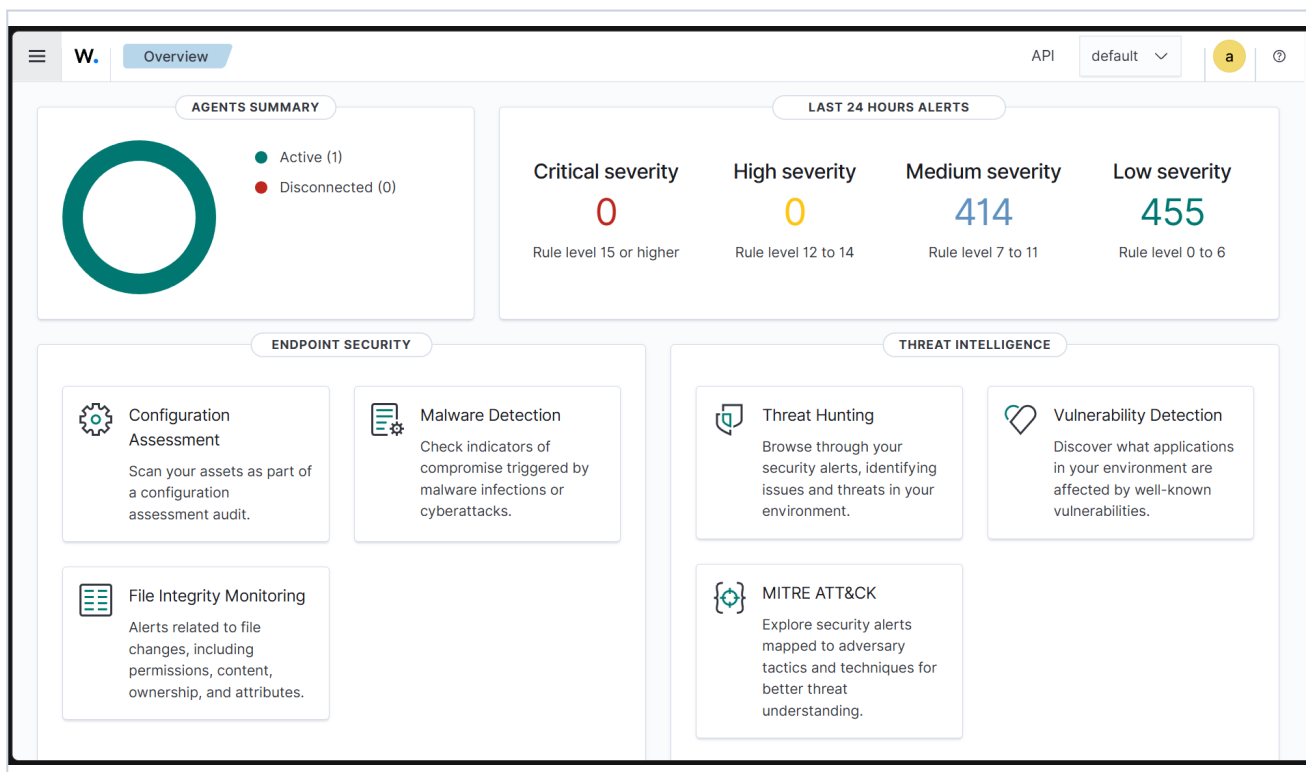


Figure 2. Wazuh Dashboard overview - one agent active, with the last-24-hours alert severity summary.

6. PowerShell Event Collection

The Wazuh Agent was configured to collect the Windows PowerShell Operational event channel, via the following block added to the agent configuration:

```
C:\Program Files (x86)\ossec-agent\ossec.conf
```

```
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

The Wazuh Agent service was restarted after the configuration change:

```
Restart-Service -Name Wazuh
```

7. PowerShell Event Validation

A controlled PowerShell command was executed to generate baseline PowerShell activity:

```
Write-Output "SOC-LAB-PowerShell-Test"
```

PowerShell Operational events were generated and collected by Wazuh. The Threat Hunting interface showed PowerShell-related events from the Windows endpoint, including Wazuh's built-in rule 91816 ("Powershell script querying system environment variables"), confirming telemetry was flowing correctly before the custom rule was introduced.

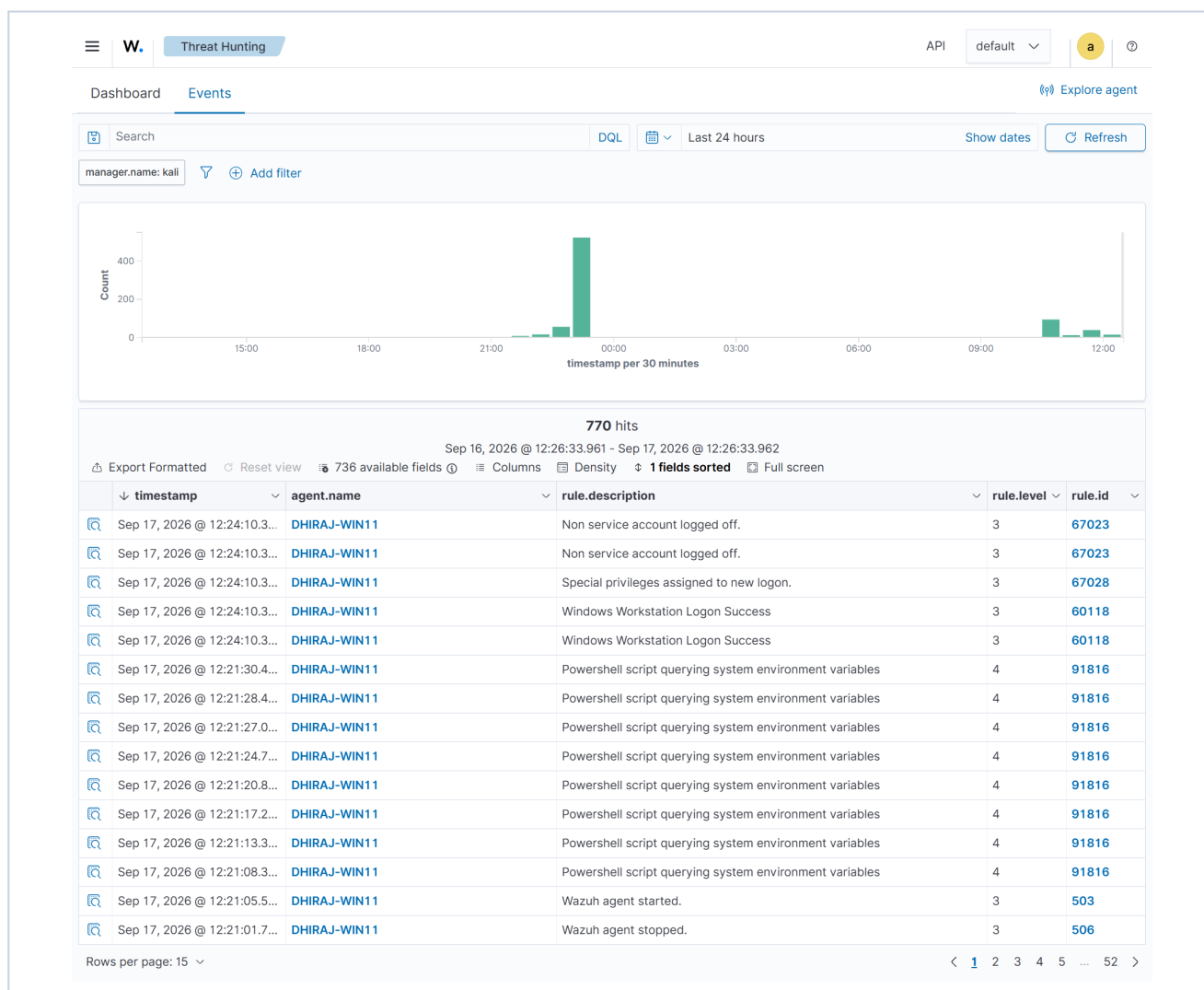


Figure 3. PowerShell and authentication events for DHIRAJ-WIN11 in Wazuh Threat Hunting - 770 hits over 24 hours, including built-in rule 91816.

8. Custom Wazuh Detection Rule

A custom detection rule was added to `/var/ossec/etc/rules/local_rules.xml`, using built-in Wazuh rule 91802 as its parent and examining the PowerShell Script Block content directly.

Detection Logic

The rule searches PowerShell Script Block text (case-insensitive) for:

```
Invoke-Expression  
IEX
```

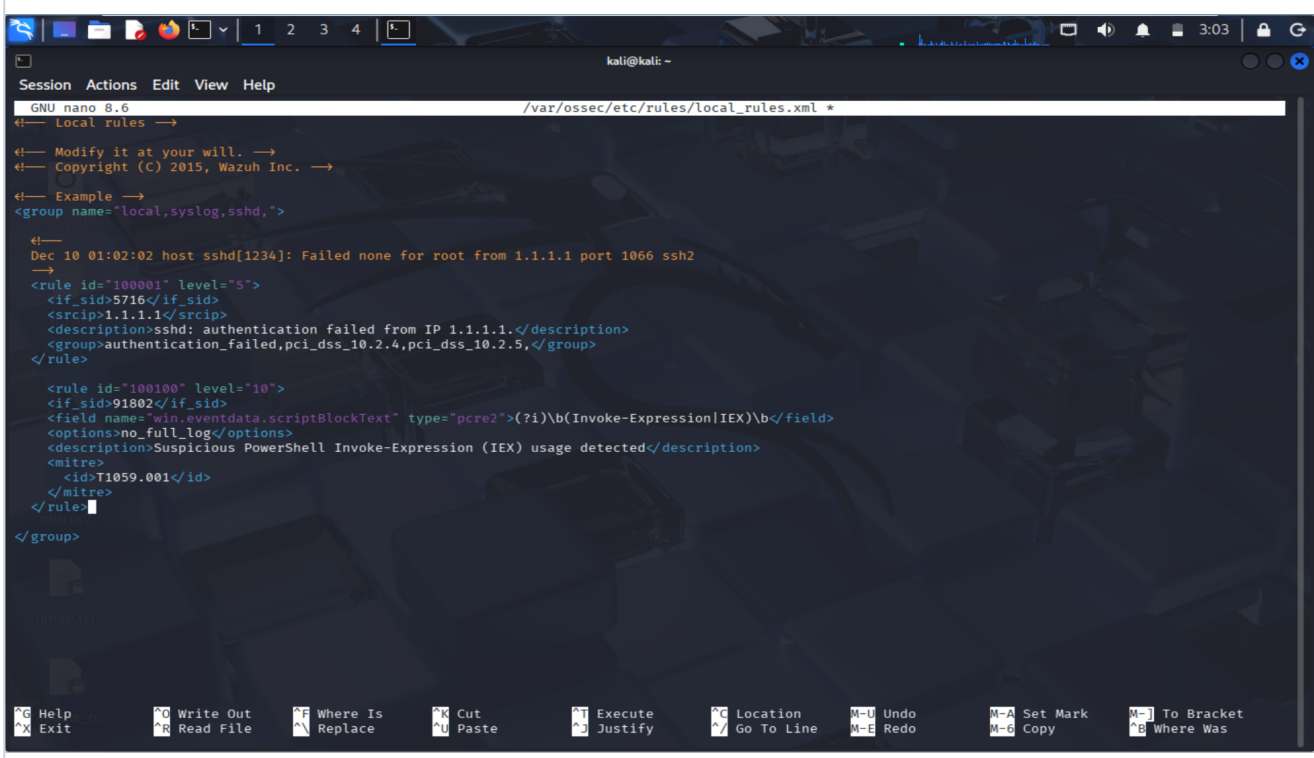
Custom Rule (local_rules.xml)

```
<rule id="100100" level="10">  
  <if_sid>91802</if_sid>  
  <field name="win.eventdata.scriptBlockText"  
    type="pcre2">(?i)\b(Invoke-Expression|IEX)\b</field>  
  <options>no_full_log</options>  
  <description>Suspicious PowerShell Invoke-Expression (IEX) usage detected</description>  
  <mitre>  
    <id>T1059.001</id>  
  </mitre>  
</rule>
```

Rule Validation

The configuration was syntax-checked and the manager restarted:

```
sudo /var/ossec/bin/wazuh-analysisd -t  
sudo systemctl restart wazuh-manager
```



```
GNU nano 8.6 /var/ossec/etc/rules/local_rules.xml *  
Local rules  
Modify it at your will.  
Copyright (C) 2015, Wazuh Inc.  
Example  
<group name="local,syslog,sshd,">  
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2  
  <rule id="100001" level="5">  
    <if_sid>5716</if_sid>  
    <srcip>1.1.1.1</srcip>  
    <description>sshd: authentication failed from IP 1.1.1.1</description>  
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>  
  </rule>  
  <rule id="100100" level="10">  
    <if_sid>91802</if_sid>  
    <field name="win.eventdata.scriptBlockText" type="pcre2">(?i)\b(Invoke-Expression|IEX)\b</field>  
    <options>no_full_log</options>  
    <description>Suspicious PowerShell Invoke-Expression (IEX) usage detected</description>  
    <mitre>  
      <id>T1059.001</id>  
    </mitre>  
  </rule>  
</group>
```

Figure 4. Custom rule 100100 defined in `local_rules.xml` on the Wazuh Manager (Kali Linux).

9. Detection Testing

A controlled PowerShell test was executed on the Windows endpoint:

```
Invoke-Expression 'Write-Output "SOC-LAB-IEX-TEST"'
```

This generated a PowerShell Script Block event that matched the custom detection rule.

Alert Details

Rule ID	100100
Level	10
Windows Event ID	4104
MITRE ATT&CK	T1059.001
Technique	PowerShell
Tactic	Execution
Agent	DHIRAJ-WIN11
Script Block	Invoke-Expression 'Write-Output "SOC-LAB-IEX-TEST"'

10. Alert Investigation

The generated alert was investigated directly in the Wazuh Dashboard. The investigation confirmed:

- The alert was generated by custom rule 100100
- The underlying Windows event was Event ID 4104
- The PowerShell Script Block contained Invoke-Expression
- The event originated from the configured Windows endpoint (agent DHIRAJ-WIN11, 192.168.47.1)
- The detection was mapped to MITRE ATT&CK T1059.001

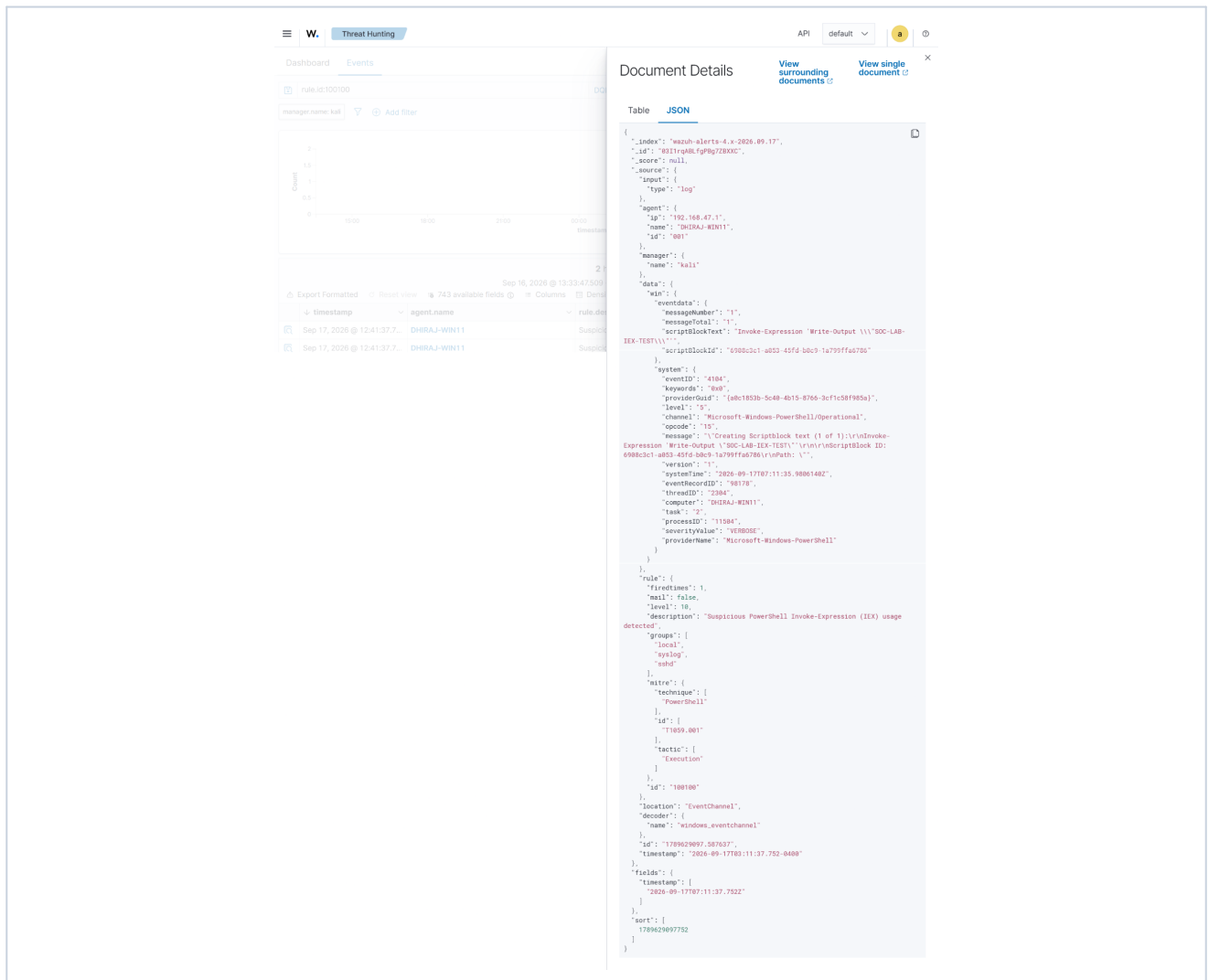


Figure 5. Full JSON alert document for the triggered rule, showing agent, script block content, event ID 4104, and the MITRE ATT&CK mapping.

The test was intentionally generated as part of SOC home-lab validation.

11. MITRE ATT&CK Mapping

ATT&CK ID	Technique	Relevance
T1059.001	Command and Scripting Interpreter: PowerShell	PowerShell activity was detected through Script Block Logging

12. Incident Disposition

Classification	Benign / Authorized Security Testing
Status	Closed
Compromise Assessment	No evidence of compromise observed during the controlled test
Remediation	No remediation required

FINAL CLASSIFICATION: BENIGN -- AUTHORIZED SECURITY TESTING

The PowerShell command was executed intentionally to validate the detection rule. No evidence was observed of:

- Malware execution
- Persistence
- Credential theft
- Unauthorized access
- System compromise

13. Detection Workflow

```
Windows PowerShell
|
v
Script Block Logging
|
v
PowerShell Event ID 4104
|
v
Wazuh Agent
|
v
Wazuh Manager
|
v
Custom Rule 100100
|
v
Alert
|
v
SOC Investigation
|
v
MITRE ATT&CK T1059.001
|
v
Incident Documentation
```

14. Skills Demonstrated

- SIEM Monitoring
- Windows Security Event Analysis
- PowerShell Logging & Script Block Logging
- Wazuh Agent Deployment & Log Collection
- Wazuh Rule Creation & Detection Engineering
- Regular-Expression-Based Detection (PCRE2)
- Alert Investigation
- MITRE ATT&CK Mapping
- Incident Classification

- SOC Investigation Workflow & Documentation

Key Learning Outcomes

This project provided hands-on experience with the complete detection-engineering lifecycle:

- 1 Generating endpoint telemetry
- 2 Enabling security logging
- 3 Collecting Windows events
- 4 Sending telemetry to a SIEM
- 5 Creating a custom detection rule
- 6 Testing the detection
- 7 Investigating the resulting alert
- 8 Mapping the activity to MITRE ATT&CK
- 9 Classifying the security event
- 10 Documenting the investigation

Lab Disclaimer: This project was performed in a controlled personal SOC home-lab environment. The PowerShell activity used for detection validation was intentionally executed for security testing and detection-engineering purposes. The alert generated during testing was classified as Benign / Authorized Security Testing. No real-world malicious activity or unauthorized access was performed.

15. Project Status: Completed

[x] PowerShell Logging Enabled	[x] Script Block Logging Verified
[x] Wazuh Agent Deployed	[x] PowerShell Event Collection
[x] Custom Detection Rule (100100)	[x] Rule Validated & Reloaded
[x] Detection Testing	[x] Alert Investigation
[x] MITRE ATT&CK Mapping	[x] Incident Classification
[x] Project Documentation	[x] Evidence Screenshots

16. Conclusion

This project demonstrates a complete detection-engineering workflow: enabling endpoint telemetry, deploying and configuring a Wazuh agent, writing and validating a custom detection rule from scratch, testing it against real activity, and investigating the resulting alert through to a MITRE ATT&CK-mapped, documented conclusion.

github.com/dhirajantre/wazuh-powershell-detection-engineering